SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q2 – Sorting the Examination Hall Seating Register

Implementation

```
/* every comparison increments cmp; every swap (or shift) increments mv */
void bubble(int a[], int n){ for(int i=0;i<n-1;i++){ int sw=0;
   for(int j=0;j<n-1-i;j++){ cmp++; if(a[j]>a[j+1]){ int t=a[j];a[j]=a[j+1];a[j+1]=t; mv++; sw=1; } }
   if(!sw) break; } }
void selection(int a[], int n){ for(int i=0;i<n-1;i++){ int m=i;
   for(int j=i+1;j<n;j++){ cmp++; if(a[j]<a[m]) m=j; }
   if(m!=i){ int t=a[i];a[i]=a[m];a[m]=t; mv++; } } }
void insertion(int a[], int n){ for(int i=1;i<n;i++){ int k=a[i], j=i-1;
   while(j>=0){ cmp++; if(a[j]>k){ a[j+1]=a[j]; mv++; j--; } else break; }
   a[j+1]=k; } }
```

Test inputs: sorted; nearly sorted (sorted except 20 late registrations with random numbers at the end); reverse-sorted; random. All three sorts produced a correctly sorted array on every input.

Student – Algorithm Design and Tracing (array 45 12 38 7 29 3)

Algorithm	Array after each pass / step	Counts
Bubble	P1: 12 38 7 29 3 45 · P2: 12 7 29 3 38 45 · P3: 7 12 3 29 38 45 · P4: 7 3 12 29 38 45 · P5: 3 7 12 29 38 45	15 comparisons, 12 swaps (measured)
Selection	i=0: 3 12 38 7 29 45 · i=1: 3 7 38 12 29 45 · i=2: 3 7 12 38 29 45 · i=3: 3 7 12 29 38 45 · i=4: no swap	15 comparisons, 4 swaps (measured)
Insertion	12 | 12 45 | 12 38 45 | 7 12 38 45 | 7 12 29 38 45 | 3 7 12 29 38 45	14 comparisons, 12 shifts (measured)

Early-exit flag: if a whole pass makes no swap, the array is already sorted, so the algorithm stops. Without it, bubble sort always makes n(n − 1)/2 comparisons; with it, an already sorted array needs only n − 1 (measured 9,999 for n = 10,000).

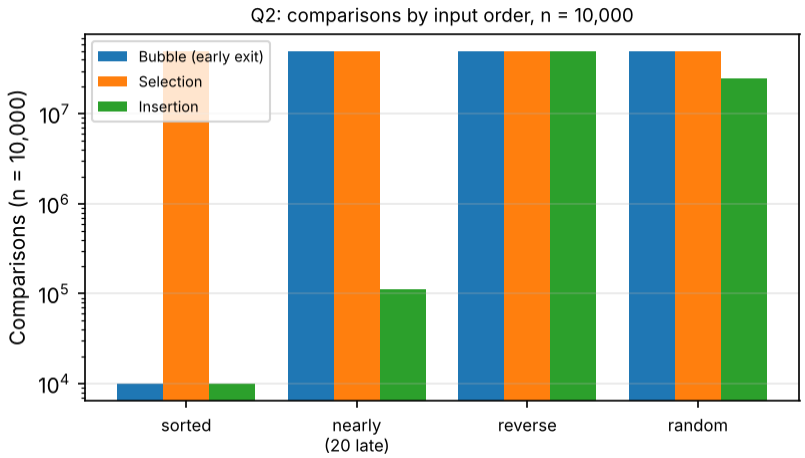
Insertion sort on nearly sorted data: each element is shifted only as far as it is out of place, so the total number of shifts equals the number of inversions. A sorted array needs n − 1 comparisons and 0 shifts; with 20 late registrations it needs about 20 × n/2 shifts (measured 101,499 for n = 10,000) instead of about $n^2/4$ = 25 million.

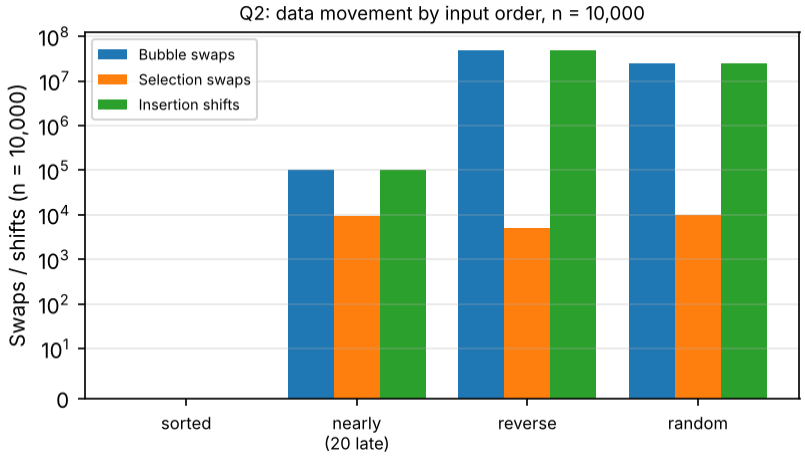
Stability: bubble sort and insertion sort (as written, with a strict > comparison) are stable; selection sort is not, because its long-distance swap can jump an element over an equal one. Measured: sorting marks 80(id1)

90(id2) 80(id3) 70(id4) 80(id5) with selection gave ids 4, 3, 1, 5, 2 – the three 80s came out as 3, 1, 5. Stability matters for students with equal marks: if the list was already sorted by name, a stable sort by marks keeps equal-mark students in name order.

Student – Comparative Measurement (comparisons / swaps or shifts)

n	Order	Bubble (early exit)	Selection	Insertion
100	sorted	99 / 0	4,950 / 0	99 / 0
100	nearly	4,895 / 728	4,950 / 98	826 / 728
100	reverse	4,950 / 4,950	4,950 / 50	4,950 / 4,950
100	random	4,859 / 2,203	4,950 / 95	2,300 / 2,203
1,000	sorted	999 / 0	499,500 / 0	999 / 0
1,000	nearly	495,672 / 9,026	499,500 / 922	10,025 / 9,026
1,000	reverse	499,500 / 499,500	499,500 / 500	499,500 / 499,500
1,000	random	498,174 / 250,027	499,500 / 993	251,020 / 250,027
10,000	sorted	9,999 / 0	49,995,000 / 0	9,999 / 0
10,000	nearly	49,911,155 / 101,499	49,995,000 / 9,586	111,498 / 101,499
10,000	reverse	49,995,000 / 49,995,000	49,995,000 / 5,000	49,995,000 / 49,995,000
10,000	random	49,990,629 / 24,947,183	49,995,000 / 9,991	24,957,172 / 24,947,183





Input order	Best algorithm	Reason
Sorted	Insertion (or bubble with early exit)	n − 1 comparisons, no moves; selection still makes n(n − 1)/2
Nearly sorted	Insertion	111,498 comparisons vs about 50 million for bubble and selection (n = 10,000)
Reverse	Selection if writes are expensive	only n/2 swaps; every algorithm still makes n(n − 1)/2 comparisons
Random	Insertion (fewest comparisons, about $n^2/4$); selection (fewest writes)	all are $O(n^2)$, insertion about half the comparisons

Result that differs from expectation: bubble sort with an early-exit flag was expected to do well on nearly sorted data, but needed 49.9 million comparisons (almost as many as on random data). The 20 late registrations were random, so some small numbers sit at the end, and bubble sort moves a small element only one place to the left per pass; the flag cannot stop the passes until all 20 have reached their places. Insertion sort moves each one directly.

Joint conclusion: for a register sorted except for 20 late registrations, use insertion sort: it needs about 111,498 comparisons for n = 10,000 against about 50 million for bubble or selection, it is stable, and it needs no extra memory. An even better plan for a big hall is to sort only the 20 new entries (a tiny cost) and merge them into the sorted register in O(n + 20) (see Q3).

Alternative Concept and Complexity Analysis – counting sort

```
int cnt[K] = {0};                    /* register numbers 0..K-1 */
for (int i = 0; i < n; i++) cnt[a[i]]++;
int p = 0;
for (int v = 0; v < K; v++)
   while (cnt[v]--) a[p++] = v;      /* write each value back as many times as it occurred */
```

Time O(n + k) and extra space O(k), against $O(n^2)$ for the three simple sorts. Measured example: n = 1,000 and k = 3,000 give about 4,000 operations instead of 1,000,000 comparisons. It is not suitable when the range k is huge compared with n (e.g. 10-digit register numbers for 100 students would need an array of 10^10 entries), when keys are not integers (names, decimals) or are negative or very sparse, or when memory is tight. If whole records must be sorted stably, the prefix-sum version of counting sort (placing records from the back) is needed instead of the simple version above.